

Internet of Things

Projeto – Controlo de rega

Alunos

Daniel Guerreiro a37379

Liliana Paquete a43720

índice

Introdução.....	3
Desenvolvimento	4
Sensores	Erro! Marcador não definido. 5
Código	6,7,8,9,10 e 11
NODE-REDE	12
InfluxDB, Grafana e aplicação MQTT	13
Conclusão	Erro! Marcador não definido. 4

Introdução

A necessidade de incremento da produção agrícola tem acelerado a inclusão de tecnologias emergentes para a monitorização da saúde das plantas, principalmente tecnologias relacionadas a Internet das Coisas (IdC).

Este trabalho tem como objetivo aplicar conhecimentos adquiridos nas aulas teóricas e práticas, para isso usamos um ESP32 com sensores para monitorizamos os dados de temperatura, humidade do ar e humidade do solo de uma planta, medir a amperagem e tensão da bateria, para quando necessário fazermos a substituição da mesma, medir o caudal de água. A rega é realizada através de um botão programado na aplicação MQTT DASH e no NOD RED, que liga uma electroválvula. As leituras dos sensores são guardadas no INFLUXDB e visualizadas e analisadas nessa aplicação e no grafana.

Desenvolvimento

Para a realização deste trabalho, recorreremos a um caso real, numa plantação de morangos, em que a rega é feita, utilizando a técnica de gota-a-gota.

Neste tipo de técnica, existem alguns problemas como o entupimento das mangueiras e do filtro, para isso usamos um sensor de caudal de água, como mostra a figura 1, para ver se a electroválvula está ligada. Caso se ligue e a informação recebida seja que a “bomba está desligada” teremos de ir ao local verificar e resolver o problema.



Figura 1

Usamos o INA219, para monitorizar a amperagem e a tensão da bateria, para a substituição da mesma quando necessário, Como Mostra a figura 2 e 3.



Figura 2

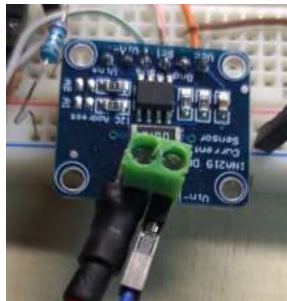


Figura 3

Para medir a temperatura e a humidade do ar usamos o sensor DH11, como se verifica na figura 4.

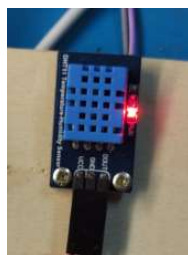


Figura 4

A medição da humidade do solo é realizada pelo sensor capacitivo da figura 5. A escolha deste sensor foi baseada no nível de corrosão, este tipo de sensor é menos corrosivo do que o sensor resistivo, por exemplo.



Figura 5

Subscrevemos à pagina do IPMA para recebermos o estado do tempo para o dia atual, para nos ajudar na decisão de rega ou não.

Todos os dados recolhidos pelos sensores irão ajudar na decisão de efetuarmos a rega ou não.

O kit de alimentação é constituído por uma bateria de 12 V de 80 A. Um inversor de 12 V DC para 220 V AC, um LED DRIVER de modelo OTM-E30-24, por este ser razoavelmente barato, alimentado por 220V AC que tem à saída 24V DC como visualizamos nas figuras 6 e 7.



Figura 6



Figura 7

O LED DRIVE vai alimentar a electroválvula da figura 8:



Figura 8

Para o controlo da electroválvula utilizamos um módulo de relé, que para ligar a electroválvula fornece-lhe 24 V e para desligar, retira-lhe a alimentação.

Para isso desenvolvemos o código seguinte:

```
projeto_JoT_final | Arduino 1.8.19
Ficheiro Editar Rascunho Ferramentas Ajuda

projeto_JoT_final$
//UTILIZAR O NODE-MCU-1.0

#include <WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <SPI.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include "DHTStable.h"
DHTStable DHT;
#define DHT11_PIN 4
#define SCREEN_WIDTH 128 // OLED display width, in pixels
#define SCREEN_HEIGHT 64 // OLED display height, in pixels
#define OLED_RESET -1 // Reset pin # (or -1 if sharing Arduino reset pin)
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
#include "Adafruit_INA219.h"
Adafruit_INA219 ina219;

const char* mqtt_server = "broker.hivemq.com";

const char* ssid = "agents";
const char* password = "Qgc908VucABYqvVu58ruv1zdpqM66ed23K64E1V7v2iJND580bzYvaHqz5k07G2";
const int LED_BUILTIN = 2; //eletrovalvul

int SensorPin = 32;
const int led_verde = 22; //Caso se queira colocar mais uma eletrovalvula
const int AirValue = 3395; //Valor do sensor sem humidade (0%)
const int WaterValue = 1300; //Valor do sensor com humidade (100%)
const int BUILTIN_LED = 2;
int soilMoistureValue = 0;

Carregamento completo
Leaving...
Hard resetting via RTS pin...

ESP32 Dev Module Disabled Default 4MB with esp32 11 2MB AP Pin 5MB SPIFFS, 240MHz (WiFi/BT), 910, 80MHz 4MB (32MB), 521600, Nome em COM2
2305
10/06/2022
```

```
projetoIoT_final | Arduino 1.8.19
Ficheiro Editor Rascunho Ferramentas Ajuda

projetoIoT_final$

int soilmoisturepercent = 0;

//
const int portaVazao = GPIO_NUM_23;
static void atualizaVazao();
volatile int pulsos_vazao = 0;
float vazao = 0;
// interrupção
void IRAM_ATTR gpio_isr_handler_up(void* arg)
{
    pulsos_vazao++;
    portYIELD_FROM_ISR();
}

WiFiClient espClient;
PubSubClient client(espClient);
// Mensagens necessária para publicar
long lastTime = 0;
const int Msg_Size6 = 50;
char msg6[Msg_Size6];
const int Msg_Size1 = 50;
char msg1[Msg_Size1];
const int Msg_Size2 = 50;
char msg2[Msg_Size2];
const int Msg_Size3 = 50;
char msg3[Msg_Size3];
const int Msg_Size4 = 50;
char msg4[Msg_Size4];
const int Msg_Size5 = 50;
char msg5[Msg_Size5];

int value = 0;

Carregamento completo
Leaving...
Hard resetting via RTS pin...

ESP32 Dev Module, Disabled, Default 4MB with spiffs (1.2MB APP+1.5MB SPIFFS), 240MHz (WiFiBT), QIO, 80MHz 4MB (32MB), 521600, Nome em COM2
```

```
projetoIoT_final | Arduino 1.8.19
Ficheiro Editor Rascunho Ferramentas Ajuda

projetoIoT_final$

WiFiClient espClient;
PubSubClient client(espClient);
// Mensagens necessária para publicar
long lastTime = 0;
const int Msg_Size6 = 50;
char msg6[Msg_Size6];
const int Msg_Size1 = 50;
char msg1[Msg_Size1];
const int Msg_Size2 = 50;
char msg2[Msg_Size2];
const int Msg_Size3 = 50;
char msg3[Msg_Size3];
const int Msg_Size4 = 50;
char msg4[Msg_Size4];
const int Msg_Size5 = 50;
char msg5[Msg_Size5];

int value = 0;

const int INA_addr = 0x40; // INA219 address
// set the INA219 to address 0x40
// indica o endereço do INA219 - 0x40

void setup_wifi() {
    Serial.println("Starting Setup Wifi");
    delay(10);
    // We start by connecting to a WiFi network
    Serial.println();
    Serial.print("Connecting to Wifi ");
    while (WiFi.status() != WL_CONNECTED) {
        delay(5000);
        Serial.print(".");
    }
    Serial.println("\nWiFi connected");
}

Carregamento completo
Leaving...
Hard resetting via RTS pin...

ESP32 Dev Module, Disabled, Default 4MB with spiffs (1.2MB APP+1.5MB SPIFFS), 240MHz (WiFiBT), QIO, 80MHz 4MB (32MB), 521600, Nome em COM2
```

```
projetoIoT_final | Arduino 1.8.19
Ficheiro Editar Rascunho Ferramentas Ajuda

projetoIoT_final$

void setup_wifi() {
  Serial.println("Starting Setup Wifi");
  delay(10);
  // We start by connecting to a Wifi network
  Serial.println();
  Serial.print("Connecting to Wifi ");
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(3000);
    Serial.print(".");
  }
  randomSeed(micros());
  Serial.println("");
  Serial.println("WiFi connected");
}

void callback(String topic, byte* payload, unsigned int length) {

  Serial.print("Message arrived [");
  Serial.print(topic);
  Serial.print("] ");
  String strPayload="";
  for (int i=0;i<length;i++) {
    strPayload = strPayload + (char)payload[i];
  }

  Serial.println(strPayload);
  StaticJsonBuffer<200> jsonBuffer;
  JsonObject data = jsonBuffer.parseObject(strPayload);

  Carregamento completo
  Leaving...
  Hard resetting via RTS pin...
37
```

```
projetoIoT_final | Arduino 1.8.19
Ficheiro Editar Rascunho Ferramentas Ajuda

projetoIoT_final$

//liga o LED interno do esp32 e liga a eletrovalvula atravez de um relé
if(topic == "IPB/IoT/Aula02/Aluno37379/LED") {
  if(data["Detection"] == "1"){
    digitalWrite(LED_BUILTIN, HIGH); // Turn on the LED
  }else if (data["Probabilidade"] < "10"){
    digitalWrite(LED_BUILTIN, HIGH); //Turn off the LED
  }else{
    digitalWrite(LED_BUILTIN, LOW); // Turn on the LED
  }else if (data["Probabilidade"] >= "10"){
    digitalWrite(LED_BUILTIN, LOW); //Turn off the LED
  }
}

Serial.println();

}

void reconnect() {
  // Loop until we're reconnected
  while (!client.connected()) {
    Serial.print("Attempting MQTT connection...");
    // Create a random client ID
    String clientId = "AulaIoT-";
    clientId += String(random(0xffff), HEX);
    // Attempt to connect
    if (client.connect(clientId.c_str())) {
      Serial.println("connected to the Broker");
      client.subscribe("IPB/IoT/Aula02/Aluno37379/LED"); //Tópico que o ESP subscrive.
      client.subscribe("IPB/IoT/Aula02/Aluno37379/LED_VERDE"); //Tópico que o ESP subscrive.
    } else {
      Serial.print("failed, rc=");
      Serial.print(client.connectRet());
    }
  }
}
```



```
projetoIoT_final | Arduino 1.8.19
Ficheiro Editor Rascunho Ferramentas Ajuda

projetoIoT_final$
String clientId = "AulaIoT-";
clientId += String(random(0xffff), HEX);
// Attempt to connect
if(client.connect(clientId.c_str())) {
  Serial.println("connected to the Broker");
  client.subscribe("IPB/IoT/Aula02/Aluno37379/LED"); //Tópico que o ESP subscrive.
  client.subscribe("IPB/IoT/Aula02/Aluno37379/LED_VERDE"); //Tópico que o ESP subscrive.
} else {
  Serial.print("failed, rc=");
  Serial.print(client.state());
  Serial.println("try again in 5 seconds");
  delay(5000);
}
if(client.connect(clientId.c_str())) {
  Serial.println("connected to the Broker");
  client.subscribe("IPB/IoT/Aula02/Aluno37379/LED_VERDE"); //Tópico que o ESP subscrive.
} else {
  Serial.print("failed, rc=");
  Serial.print(client.state());
  Serial.println("try again in 5 seconds");
  delay(5000);
}
}

//Inicializa sensor o caudalimetro com interrupção na subida de um pulso
void iniciaVazao(gpio_num_t Port){
  gpio_set_direction(Port, GPIO_MODE_OUTPUT);
  gpio_set_intr_type(Port, GPIO_INTR_NEGEDGE);
  gpio_set_pull_mode(Port, GPIO_PULLUP_ONLY);
  gpio_intr_enable(Port);
  gpio_install_isr_service(0);
  gpio_isr_handler_add(Port, gpio_isr_handler_callback, NULL);
}

Carregamento completo
Leaving...
Hard resetting via RTS pin...

ESP32 Dev Module, Disabled Default 4MB with spiffs (1.2MB APP+1.5MB SPIFFS), 240MHz (WiFi/BT), QIO, 80MHz 4MB (32MB), 521600, Nome em COM2
```

```
projetoIoT_final | Arduino 1.8.19
Ficheiro Editor Rascunho Ferramentas Ajuda

projetoIoT_final$
}

void setup() {
  Serial.begin(9600);
  pinMode(LED_BUILTIN, OUTPUT); // Inicia to LED_BUILTIN como pin de saída
  /*pinMode(led_verde, OUTPUT); // Inicia to led_verde como pin de saída, caso queira acionar mais uma eletrovalvula*/
  Serial.println("\n Start Setup");
  setup_wifi();
  client.setServer(mqtt_server, 1883);
  client.setCallback(callback);

  // definir porta do sensor do caudalimetro como entrada
  iniciaVazao((gpio_num_t) portaVazao);

  if (! ina219.begin()) {
    Serial.println("Failed to find INA219 chip");
    while (1) { delay(10); }
  }
  // Inicia a leitura da tensão, corrente e potência
  Serial.print("SV"); Serial.print("\t"); // Bus Voltage
  Serial.print("SV"); Serial.print("\t"); // Shunt Voltage
  Serial.print("LV"); Serial.print("\t"); // Load Voltage
  Serial.print("C"); Serial.print("\t"); // Current
  Serial.println("P"); // Power
}

void loop() {
  while(WiFi.status() != WL_CONNECTED) { //Check Wifi connection
    WiFi.reconnect(); //If disconnected, try to reconnect
  }
  if(!client.connected()) { //Check Check Broker connection
    Serial.println("Node disconnected of Broker. Trying to connect.. ");
    //delay(10); //Teste conexão ao Broker
  }
}
```

```
projetoIoT_final | Arduino 1.8.19
Ficheiro Editar Rascunho Ferramentas Ajuda

projetoIoT_final$

// Lê o sensor de Humidade e converte-o para percentagem

soilMoistureValue = analogRead(SensorPin); //put Sensor insert into soil
Serial.println(soilMoistureValue);
soilmoisturepercent = map(soilMoistureValue, AirValue, WaterValue, 100, 0);
delay(1000);
if (soilmoisturepercent > 100){
  client.publish("IPB/IoT/Aula02/Aluno37379/Humidade/Solo","Solo Seco");
  digitalWrite(LED_BUILTIN, HIGH); // Turn on the LED
}else if (soilmoisturepercent < 70){
  client.publish("IPB/IoT/Aula02/Aluno37379/Humidade/Solo","Solo Encharcado");
  digitalWrite(LED_BUILTIN, LOW); // Turn on the LED
}else if (soilmoisturepercent >= 0 && soilmoisturepercent <= 100){
  client.publish("IPB/IoT/Aula02/Aluno37379/Humidade/Solo","Solo Humido");
}

// Leitura de dados do DHT11
Serial.print("Humidade do ar: \t");
int chk = DHT.read11(DHT11_PIN);
// DISPLAY DATA
Serial.print(DHT.getHumidity(), 1);
Serial.print("\n");
Serial.print("Temperatura: \t");
Serial.println(DHT.getTemperature(), 1);
Serial.print("\n");
delay(2000);

// Publicações dos vários tópicos
sprintf(msg6, Msg_Size6, "%ld", soilMoistureValue);
client.publish("IPB/IoT/Aula02/Aluno37379/Humidade/Solo",msg6);

sprintf(msg1, Msg_Size1, "%.2f", DHT.getHumidity());
client.publish("IPB/IoT/Aula02/Aluno37379/Humidade/Ar",msg1);

Carregamento completo
Leaving...
Hard resetting via RTS pin...

232 ESP32 Dev Module, Disabled Default 4MB with spiffs (1.2MB APP+1.5MB SPIFFS), 240MHz (WiFi/BT), QIO, 80MHz 4MB (32Mb), 521600, Nome em COM2
```

```
projetoIoT_final | Arduino 1.8.19
Ficheiro Editar Rascunho Ferramentas Ajuda

projetoIoT_final$

// Publicações dos vários tópicos
sprintf(msg6, Msg_Size6, "%ld", soilMoistureValue);
client.publish("IPB/IoT/Aula02/Aluno37379/Humidade/Solo",msg6);

sprintf(msg1, Msg_Size1, "%.2f", DHT.getHumidity());
client.publish("IPB/IoT/Aula02/Aluno37379/Humidade/Ar",msg1);

sprintf(msg2, Msg_Size2, "%.2f", DHT.getTemperature());
client.publish("IPB/IoT/Aula02/Aluno37379/Temperatura",msg2);

// Transformação dos pulsos para correspondente caudalímetro
vazao = pulsos_vazao/5.5;
pulsos_vazao = 0;
// Realizar o print da leitura no serial
Serial.println("Leitura do Sensor de Vazao:");
Serial.println(vazao);
// realizar um delay e inicializar leitura daqui a 1 segundos
delay(1000);
if(vazao > 0 ){
  client.publish("IPB/IoT/Aula02/Aluno37379/Bomba","Ligada");
}else{
  client.publish("IPB/IoT/Aula02/Aluno37379/Bomba","Desligada");
}

if (soilmoisturepercent < 70){
  digitalWrite(LED_BUILTIN, HIGH); //Turn off the LED
}else{
}

Carregamento completo
Leaving...
Hard resetting via RTS pin...

232 ESP32 Dev Module, Disabled Default 4MB with spiffs (1.2MB APP+1.5MB SPIFFS), 240MHz (WiFi/BT), QIO, 80MHz 4MB (32Mb), 521600, Nome em COM2
```

```
projetoIoT_final | Arduino 1.8.19
Ficheiro Editar Rascunho Ferramentas Ajuda

projetoIoT_final$

// Publicações dos vários tópicos
sprintf(msg6, "%d", soilMoistureValue);
client.publish("IPB/IoT/Aula02/Aluno37379/humidade/Solo",msg6);

sprintf(msg1, "%2f", DHT.getHumidity());
client.publish("IPB/IoT/Aula02/Aluno37379/humidade/Ar",msg1);

sprintf(msg2, "%2f", DHT.getTemperature());
client.publish("IPB/IoT/Aula02/Aluno37379/Temperatura",msg2);

// Transformação dos pulsos para correspondente caudalimetro
vazao = pulsos_vazao/5.5;
pulsos_vazao = 0;
// Realizar o print da leitura no serial
Serial.println("Leitura do Sensor de Vazao:");
Serial.println(vazao);
// realizar um delay e inicializar leitura daqui a 1 segundos
delay(1000);
if(vazao > 0 ){
  client.publish("IPB/IoT/Aula02/Aluno37379/Bomba","Ligada");
}
else{
  client.publish("IPB/IoT/Aula02/Aluno37379/Bomba","Desligada");
}

if (soilmoisturepercent < 70){
  digitalWrite(LED_BUILTIN, HIGH); //Turn off the LED
}
else{
  digitalWrite(LED_BUILTIN, LOW);
}

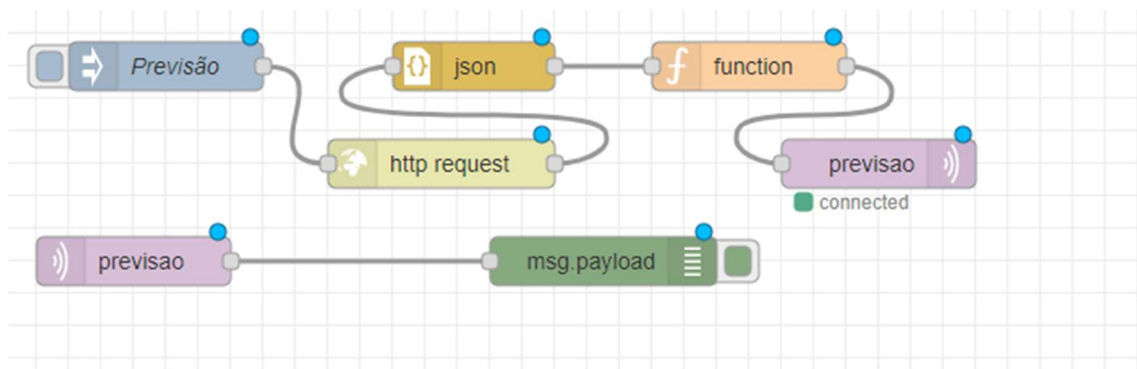
Carregamento completo
Leaving...
Hard resetting via RTS pin...

ESP32 Dev Module, Disabled, Default 4MB with spiffs (1.2MB APP+1.5MB SPIFFS), 240MHz (Vf1.0T), QIO, 80MHz 4MB (32MB), 521600, Nome em COM3
```

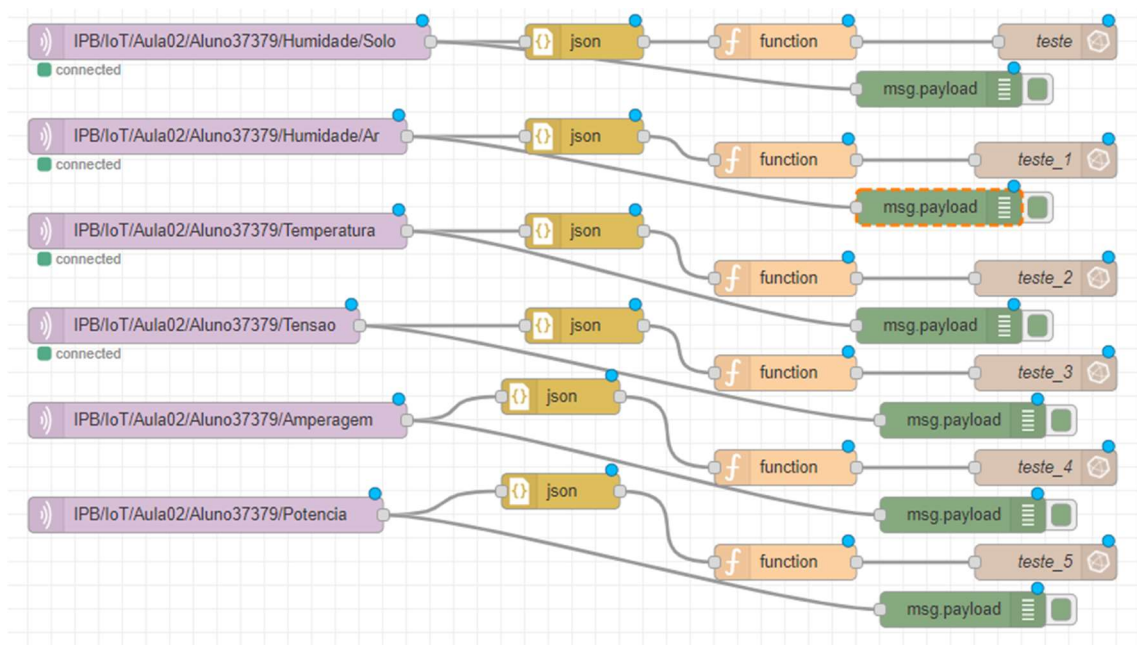
Usamos o NOD-RED para manipulação dos nós e para fazermos a ponte com o InfluxDB como mostram as figuras seguintes:

NODE-RED

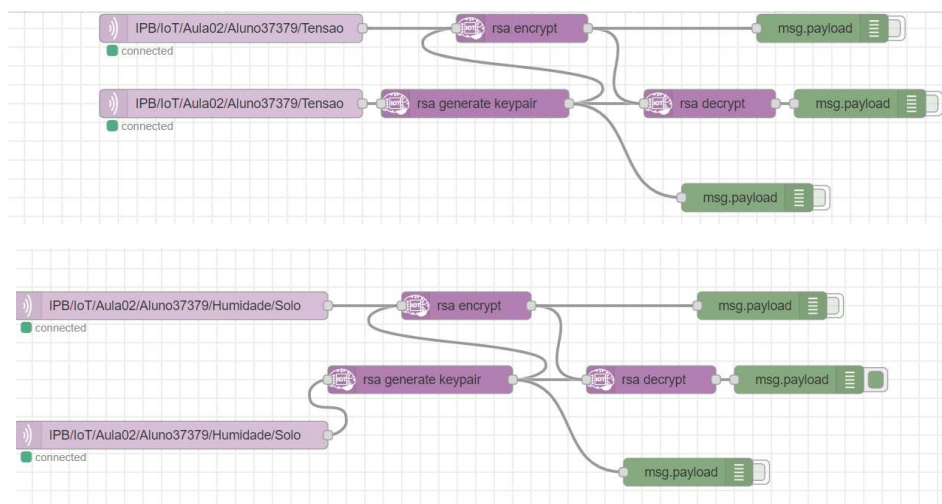
Nó da previsão



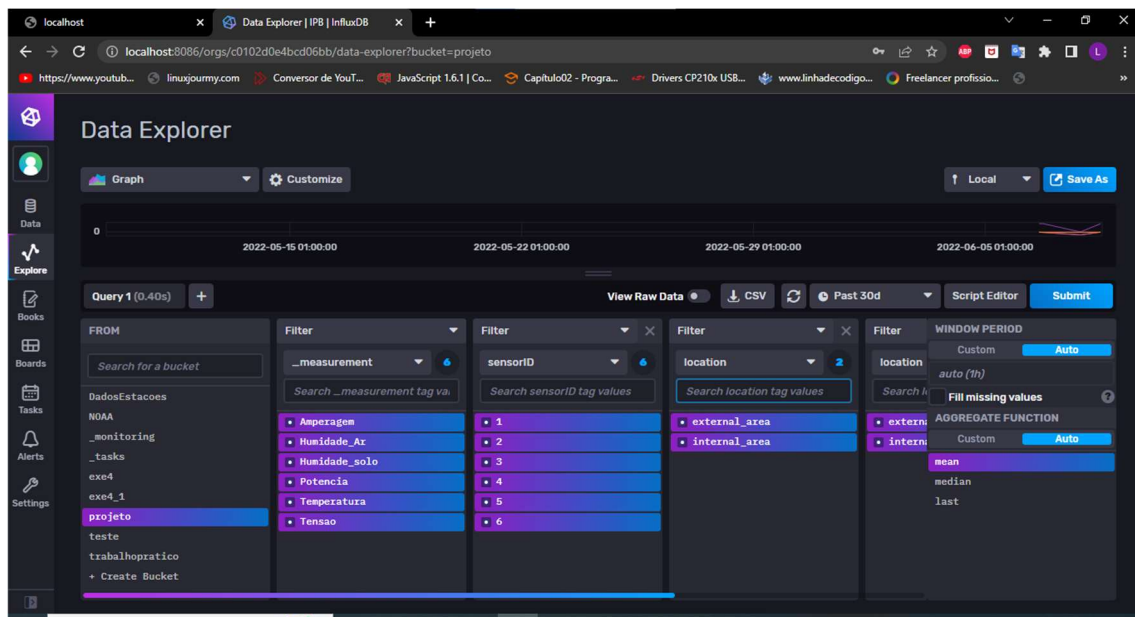
Nós enviados para a base de dados InfluxDN



Encriptação



Visualização dos dados no InfluxDB



Visualização dos dados no Grafana



Visualização dos dados na aplicação MQTT Dash



Conclusão

Neste trabalho conseguimos ultrapassar algumas dificuldades que tivemos inicialmente, como colocar os sensores todos num código só, mas também tivemos dificuldades, como ligar e desligar o botão juntamente com o sensor de humidade, porque quando a electroválvula ligava pelo sensor de humidade, o botão desligava-a e tornava a ligar novamente, por isso decidimos colocar o sensor de humidade quando queríamos rega automática e quando queríamos com o botão teríamos de voltar a configurar o ESP32 so com o botão.

Mesmo assim consegui realizar um projeto pessoal porque eu queria controlar a rega com um botão através de uma aplicação.